

# Analyse und Implementierung der Optimierungsheuristik Digital Annealing

Lino Bugia

25. November 2025

# Inhaltsverzeichnis

Kontext Optimierung

Problemcodierung als Pseudo Boolsche Funktionen

Markow Ketten Perspektive

Implementation

## Kontext Optimierung

Problemcodierung als Pseudo Boolsche Funktionen

Markow Ketten Perspektive

Implementation

# Optimierungsprobleme

## Definition

Ein Optimierungsproblem besteht aus einer Menge  $M$  und wohldefinierten Funktion  $f : M \rightarrow \mathbb{R}$ .

Gesucht sind die minimalen/maximalen Elemente:  $\min_{x \in M} f(x)$

Bemerkungen:

- ▶ Wegen:  $\max_{x \in M} f(x) \iff \min_{x \in M} -f(x)$  genügt Minimierung.
- ▶ Gleiche Probleme können auf unterschiedliche Weise modelliert werden  $\implies$  Problemtransformationen.
- ▶ Ein festes  $(M, f)$  nennt man Instanz eines Optimierungsproblems.



# Kombinatorische Optimierungsprobleme

## Definition

Ein kombinatorisches Optimierungsproblem (COP) ist ein Optimierungsproblem mit einer **endlichen** Menge  $M$ .

Beispiele:

- ▶ Zahlenpartitionierung:  
Teile Zahlen  $Z_1, \dots, Z_n$  in 2 Gruppen auf, sodass Differenz der Gruppen minimal ist.
- ▶ Travelling Salesman Problem:  
Gegeben:  $(1, \dots, n)$  Knoten, Kantenmenge  $E \subset \mathcal{P}((1, \dots, n))$ ,  $d_e \in \mathbb{R} \forall e \in E$ .  
Gesucht: Hamilton Kreis mit minimaler Distanz.
- ▶ Ganzzahlige lineare Optimierung (ILP)

$$\min_{x \in \mathbb{Z}^n} c^T x, Ax = b, g \geq x \geq 0, A \in \mathbb{R}^{m \times n}, b \in \mathbb{R}^m, g \in \mathbb{R}$$

## Lösung kombinatorischer Optimierungsprobleme mit Verfahren für Kontinua?

z.B. LP-Relaxion: Erweiterung des ILP von  $x \in \mathbb{Z}^n$  auf LP  $x \in \mathbb{R}^n$ .

Beispiel (Skript KOP, S.Körkel):

Ganzzahliges LP  $x \in \mathbb{Z}^n$ :

$$\max_{(x,y) \in \mathbb{Z}^2} x + 0.64y$$

$$50x + 31y \leq 250$$

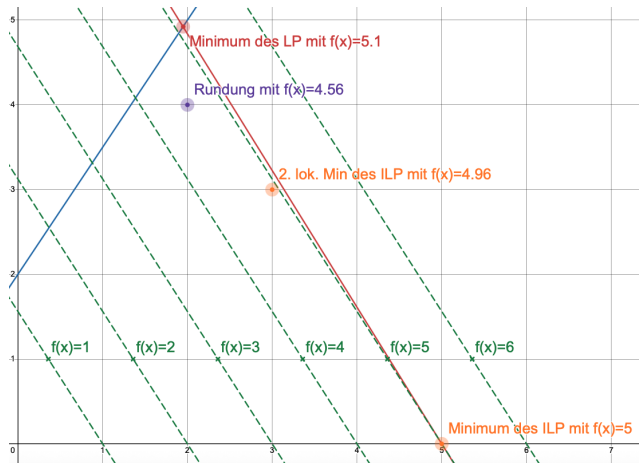
$$3x - 2y \leq -4$$

$$x, y \geq 0$$

LP Relaxion erzeugt

Rundungsproblem!

$\Rightarrow$  Nicht im Allgemeinen mit  
Verfahren für Kontinua lösbar.







## Vollständige Enumeration

Größe der Mengen M von COPs abhängig von der Zahl der Variablen:

- ▶ Travelling Salesman Problem: Menge M ist die Anzahl der Hamiltonkreise auf einem Graphen mit n Städten:  $\frac{(n-1)!}{2}$  Möglichkeiten.  
Instanz mit 100 Städte  $\Rightarrow \frac{(99)!}{2} = 9.332622 \cdot (10)^{155}$  Möglichkeiten
- ▶ Integer Linear Programm in Ungleichungsform:

$$Ax \geq b, \min_{x \in \mathbb{Z}^n} c^T x, A \in \mathbb{R}^{m \times n}, b \in \mathbb{R}^m$$

Zulässige Bereiche sind Teilmengen des  $\mathbb{R}^n$ . Erhöhung von  $n$  ergibt exponentielle Steigerung möglicher zulässiger Punkte.

- ▶ Zahlenpartitionierung: Anzahl an Unterteilungen von  $n$  Zahlen in 2 Gruppen.  $2^n$  Möglichkeiten.

Erzeugt Bottleneck für viele Variablen! Für Probleme mit mehr Variablen muss exponentiell mehr Iterationen und damit Rechenzeit gewährleistet werden.





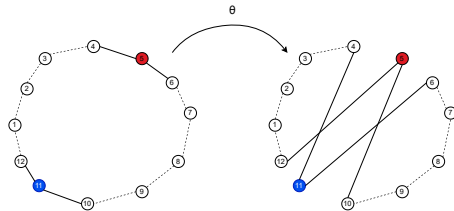
## Beispiele für Modifikationen auf dem Travelling Salesman Problem

Gegeben:  $V = \{1, \dots, n\}$  Knoten, Kantenmenge  $E \subset \mathcal{P}(\{1, \dots, n\})$ ,  $d_e \in \mathbb{R} \forall e \in E$ .

Gesucht: Hamilton Kreis  $t = (t_1, \dots, t_n)$  mit  $t_i \in \{1, \dots, n\}$  mit minimaler Distanz:

$$\min_t \sum_{i=1}^{n-1} d_{(t_i, t_{i+1})} + d_{t_n, t_1}$$

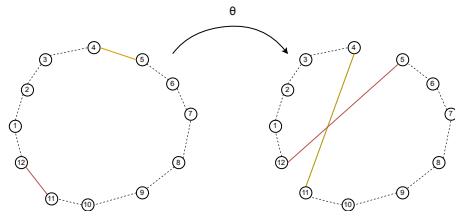
Städtetausch Modifikation:  $\theta_{(a,b)}(t)$   
Vertausche zwei Städte  $(a, b)$  mit  $a \neq b$ .



$$\Delta f(t)_{\theta_{(a,b)}} = -d_{a-1,a} - d_{a,a+1} - d_{b-1,b} - d_{b,b+1} + d_{a,b-1} + d_{a,b+1} + d_{a-1,b} + d_{a+1,b}$$

K besteht aus Paaren von Zahlen, also  
 $a \neq b \in \{1, \dots, n\} \implies |K| = \binom{n}{2} = \frac{n^2+n}{2}$

Routentausch Modifikation:  $\rho_{(a_1,a_2),(b_1,b_2)}(t)$   
Vertausche zwei Kanten  $(a_1, a_2)$  und  $(b_1, b_2)$  mit  $a_1 \neq a_2, b_1 \neq b_2$  und  $(a_1, a_2), (b_1, b_2) \in t$ .



$$\Delta f(t)_{\rho_{(a_1,a_2),(b_1,b_2)}} = -d_{a_1,a_2} - d_{b_1,b_2} + d_{a_1,b_2} + d_{a_2,b_1}$$

K besteht aus 2 Routenindizes der Tour  $t$ , also  
 $r_1 \neq r_2 \in \{1, \dots, n\} \implies |K| = \binom{n}{2} = \frac{n^2+n}{2}$

## Greedy Algorithmus:

Berechne  $\Delta f(x)_{\theta_k}$  für zufälliges  $k \in K$  und iteriere  $x_{\text{neu}} = \theta_k(x)$ , wenn  $\Delta f(x)_{\theta_k} < 0$ .

---

---

```
1: function GREEDY ALGORITHMUS( $f, M, \theta, K$ )
2:   Generiere  $x \in M$ , Berechne  $f(x)$                                 ▷ Startinitialisierung
3:   for  $i = 1$  to steps do
4:     Wähle zufälliges  $k \in K$                                         ▷ Zufallsauswahl  $k \in K$ 
5:     Berechne  $\Delta f(x)_{\theta_k}$                                     ▷ Funktionsauswertung
6:     if  $\Delta f(x)_{\theta_k} < 0$  then
7:        $x \leftarrow \theta_k(x)$                                         ▷ Iterationsschritt
8:        $f(x) \leftarrow f(x) + \Delta f(x)_{\theta_k}$                   ▷ Iterationsschritt
9:     end if
10:  end for
11:  return  $x, f(x)$ 
12: end function
```

---

Falls  $x$  lokales Minimum, das heißt  $\forall k \in K : \Delta f(x)_{\theta_k} > 0 \implies$  Keine Bewegung mehr möglich

Erlaube Verschlechterung mit einer gewissen Wahrscheinlichkeit und reduziere diese Wahrscheinlichkeit über den Laufe des Algorithmus.

Verwende dafür das **Metropolis Kriterium** abhängig vom Faktor  $T > 0$ :

$$p_a(\Delta f(x)_{\theta k}, T) = \min\{1, \exp(-\frac{\Delta f(x)_{\theta k}}{T})\}$$

- ▶ Fall  $\Delta f(x)_{\theta k} < 0 : \exp(-\frac{\Delta f(x)_{\theta k}}{T}) > 1$   
 $\Rightarrow p_a(\Delta f(x)_{\theta k}, T) = 1$   
 Kleinere Funktionswerte werden immer akzeptiert!
- ▶ Fall  $\Delta f(x)_{\theta k} > 0 : \exp(-\frac{\Delta f(x)_{\theta k}}{T}) < 1$   
 $\Rightarrow p_a(\Delta f(x)_{\theta k}, T) = \exp(-\frac{\Delta f(x)_{\theta k}}{T})$   
 Wahrscheinlichkeit für Verschlechterung.

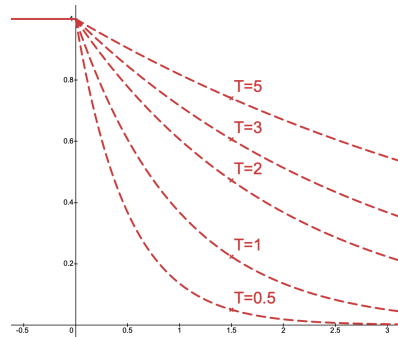


Abbildung: y Achse:  $p_a$ , x Achse  $\Delta f(x)_{\theta k}$

Erlaube Verschlechterung mit einer gewissen Wahrscheinlichkeit und reduziere diese Wahrscheinlichkeit über den Laufe des Algorithmus.

Strategie: Starte mit  $T$ , sodass Verschlechterungen erlaubt werden können. Erzeuge sogenanntes Coolingsschedule  $T_i$  mit  $T_{i+1} \leq T_i$  und  $T_i \rightarrow 0$ , wenn  $i$  das Ende Iterationen erreicht.

Beispiele für Cooling Schedules:

► Linear:

$$T_i(T_{start}, T_{end}, n) = T_{start} - \frac{i(T_{start} - T_{end})}{n}$$

mit  $T_{start} \gg T_{end} \approx 0$  und  $n$  Anzahl der Iterationen.

► Logarithmisch:

$$T_i(a, b) = \frac{a}{b \log(i)}$$

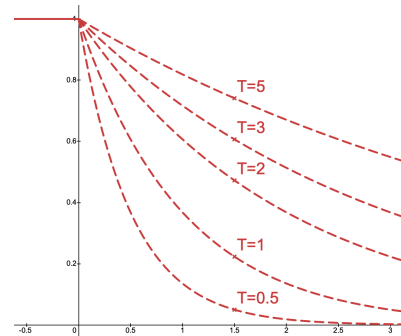


Abbildung: y Achse:  $p_a$ , x Achse  $\Delta f(x)_{\theta k}$



```

1: function SIMULATEDANNEALING( $f, M, \theta, K, T$ )
2:   Generiere  $x \in M$ , Berechne  $f(x)$  ▷ Startinitialisierung
3:   for  $i = 1$  to  $steps$  do
4:      $T \leftarrow T_i$ 
5:     Wähle zufälliges  $k \in K$  ▷ Zufallsauswahl  $k \in K$ 
6:     Berechne  $\Delta f(x)_{\theta_k}$  ▷ Funktionsauswertung
7:      $p_i = rand(0, 1)$ 
8:     if  $p_i < \min(1, \exp(-\Delta f(x)_{\theta_k}/T))$  then ▷ Metropolkriterium
9:        $x \leftarrow \theta_k(x)$  ▷ Iterationsschritt
10:       $f(x) \leftarrow f(x) + \Delta f(x)_{\theta_k}$  ▷ Iterationsschritt
11:    end if
12:  end for
13:  return  $x, f(x)$ 
14: end function

```

## Motivation für Digital Annealing

Problem bei Simulated Annealing: Wenn  $x$  nahe optimal tragen nur wenige  $k \in K$  zur Verbesserung des Funktionswerts bei. Zufällige Auswahl, die sich nach jeder korrekten Auswahl verschlechtert! Verlangsamung der Konvergenzgeschwindigkeit.

Angenommen: TSP mit  $n$  Städten, Städtetauschmodifikation,  $x$  ist eine fast optimale Route bis auf eine Modifikation der richtigen Städte  $a, b$ . Zufällige Auswahl aus  $|K| = \binom{n}{2} = \frac{n^2+n}{2} !$   
Abgeschätzt:  $|K| = \frac{n^2+n}{2}$  Iterationen notwendig (nur für eine Modifikation)!

Idee für Digital Annealing:

Berechne Differenz in der Funktion nicht nur für ein zufälliges  $k$ , sondern für alle  $k \in K$ .  
Wende auf alle Metropolkriterium an. Danach erst Zufallsauswahl der akzeptierten Menge!

---

```

1: function DIGITALANNEALING( $f, M, \theta, K, T$ )
2:   Generiere  $x \in M$ , Berechne  $f(x)$                                 ▷ Startinitialisierung
3:   for  $i = 1$  to steps do
4:      $T \leftarrow T_i$ ,  $A =$  leere Menge
5:     for  $k \in K$  do
6:       Berechne  $\Delta f(x)_{\theta_k}$                                 ▷  $|K|$  mal Funktionsauswertung
7:        $p_i = \text{rand}(0, 1)$ 
8:       if  $p_i < \min(1, \exp(-\Delta f(x)_{\theta_k}/T))$  then                ▷  $|K|$  mal Metropolis Kriterium
9:         Füge  $k$  der Menge  $A$  hinzu
10:      end if
11:    end for
12:    if  $A$  nicht leer then
13:      Wähle zufälliges  $s$  aus  $A$                                 ▷ Zufallsauswahl  $s \in A$ 
14:       $x \leftarrow \theta_s(x)$                                           ▷ Iterationsschritt
15:       $f(x) \leftarrow f(x) + \Delta f(x)_{\theta_s}$                       ▷ Iterationsschritt
16:    end if
17:  end for
18:  return  $x, f(x)$ 
19: end function

```

---

Was wenn Menge  $A$  leer ist? Dann ist  $x$  lokales Minima  $\forall k \in K : \Delta f(x)_{\theta_k} > 0$ .  $\rightarrow$  Escape Mechanismus

## Escape Mechanismus

Artifizielle Erhöhung der Verschlechterungswahrscheinlichkeiten:

- ▶  $E_{inc} > 0$  klein:  
 $\implies$  kleinere Verschlechterungen werden irgendwann (eventuell über mehrere Iterationen) zugelassen.
- ▶  $E_{inc} \gg \max_{k \in K} \Delta f(x)_{\theta_s}$ :  
 $\implies$  Alle Verschlechterungen gleich wahrscheinlich.

---

```

1: function DIGITALANNEALINGMITOFFSET( $f, M, \theta, K, T, E_{inc}$ )
2:   Generiere  $x \in M$ , Berechne  $f(x)$                                 ▷ Startinitialisierung
3:    $E_{Off} = 0$ 
4:   for  $i = 1$  to steps do
5:      $T \leftarrow T_i, A =$  leere Menge
6:     for  $k \in K$  do
7:       Berechne  $\Delta f(x)_{\theta_k}$                                 ▷  $|K|$  mal Funktionsauswertung
8:        $p_i = \text{rand}(0, 1)$ 
9:       if  $p_i > \min(1, \exp((-\Delta f(x)_{\theta_k} - E_{Off})/T))$  then
10:        Füge  $k$  der Menge  $A$  hinzu                                ▷  $|K|$  mal Metropolis Kriterium
11:      end if
12:    end for
13:    if  $A$  nicht leer then
14:      Wähle zufälliges  $s$  aus  $A$                                 ▷ Zufallsauswahl  $s \in A$ 
15:       $x \leftarrow \theta_s(x)$                                         ▷ Iterationsschritt
16:       $f(x) \leftarrow f(x) + \Delta f(x)_{\theta_s}$                     ▷ Iterationsschritt
17:       $E_{Off} = 0$ 
18:    else
19:       $E_{Off} = E_{Off} + E_{inc}$ 
20:    end if
21:  end for
22:  return  $x, f(x)$ 
23: end function

```

---



## Kontext Optimierung

## Problemcodierung als Pseudo Boolsche Funktionen

## Markow Ketten Perspektive

## Implementation

# Pseudo boolean functions

## Definition

Eine Pseudo Boolesche Funktion (PBF)  $\mathcal{P}$  von Grad  $d$  ist ein multivariates Polynom mit Variablen  $x_1, \dots, x_n \in \mathbb{F}_2$  und Koeffizienten  $a_S \in \mathbb{R}$  für die Kombinationen der Variablen  $S \subseteq \{1, \dots, n\}$  mit  $|S| \leq d$ :

$$\mathcal{P} : \mathbb{F}_2^n \longrightarrow \mathbb{R}$$

$$\mathcal{P}(x_1, x_2, \dots, x_n) = \sum_{\substack{S \subseteq \{1, \dots, n\}: \\ |S| \leq d}} a_S \prod_{k \in S} x_k.$$

Ein Paar  $(S, a_S)$  mit  $S \subseteq \{1, \dots, n\}$  nennen wir ein Monom  $S$  von  $\mathcal{P}$ . Die Menge der Monome eines PBF  $\mathcal{P}$  ist  $\text{Mon}(\mathcal{P})$ .

Kombinatorische Optimierungsprobleme  $(\mathcal{P}, \mathbb{F}_2^n)$  heißen polynomial unconstrained binary optimization (PUBO).

$$\mathcal{P}(x_1, x_2, \dots, x_n) = \sum_{\substack{S \subseteq \{1, \dots, n\}: \\ |S| \leq d}} a_S \prod_{k \in S} x_k$$

Es gilt:

- ▶  $\forall x \in \mathbb{F}_2 \wedge \forall n \in \mathbb{N} : x^n = x$
- ▶ Anzahl von Monomen in einem PBF mit Grad  $d$

$$|\text{Mon}(\mathcal{P})| = \sum_{i=0}^d \binom{n}{i}$$

- ▶ Dichte des PBF:

$$\delta(\mathcal{P}) = \frac{|\{(S, a_S) \in \text{Mon}(\mathcal{P}) : a_S \neq 0\}|}{|\text{Mon}(\mathcal{P})|}$$

- ▶ Über die bijektive Abbildung  $\Theta : \{-1\}^n \rightarrow \mathbb{F}_2^n$ :

$$\Theta(s_1, \dots, s_n) = \left( \frac{s_1 + 1}{2}, \dots, \frac{s_n + 1}{2} \right)$$

lässt sich auch der Input  $\{-1\}^n$  für das PBF verwenden:  $\mathcal{P}(\Theta(s))$ .



## Die Bitflip Modifikation

Der Punkt  $x = (x_1, \dots, x_n)^T \in \mathbb{F}_2^n$  wird abgebildet auf

$$\beta_k(x) = (x_1, \dots, x_{k-1}, x_k \oplus 1, x_{k+1}, \dots, x_n)^T.$$

mit der Restklassenaddition auf  $\mathbb{F}_2$ :

$$0 \oplus 1 = 1$$

$$1 \oplus 1 = 0$$

Beschreibung des Bitflip:

$$\Delta P(x)_{\beta_k} = (-1)^{(x_k)} \sum_{\substack{(S, a_S) \in \text{Mon}(\mathcal{P}): \\ k \in S}} a_S \prod_{\substack{i \in S \\ i \neq k}} x_i$$

# Beweis:

Modifikationseigenschaft?  $\longrightarrow$  klar, jeder Zustand mit maximal  $n$  Modifikationen erreichbar.

Mit der Monommengende des PBF  $\text{Mon}(\mathcal{P})$  lässt sich das PBF beschreiben, als

$$\mathcal{P}(x) = \sum_{(S, a_S) \in \text{Mon}(\mathcal{P})} a_S \prod_{i \in S} x_i$$

Differenz der Ausdrücke:

$$\Delta \mathcal{P}(x)_{\beta_k} = \mathcal{P}(\beta_k(x)) - \mathcal{P}(x) = \sum_{(S, a_S) \in \text{Mon}(\mathcal{P})} a_S \prod_{i \in S/\{k\}} x_i \prod_{i \in S \cap \{k\}} (x_k \oplus 1) - \left( \sum_{(S, a_S) \in \text{Mon}(\mathcal{P})} a_S \prod_{i \in S} x_i \right)$$

Monome  $(S, a_S)$  mit  $k \notin S$  kürzen sich weg und faktorisieren danach  $x_k$  aus

$$\begin{aligned} & (x_k \oplus 1) \left( \sum_{\substack{(S, a_S) \in \text{Mon}(\mathcal{P}): \\ k \in S}} a_S \prod_{\substack{i \in S \\ i \neq k}} x_i \right) - x_k \left( \sum_{\substack{(S, a_S) \in \text{Mon}(\mathcal{P}): \\ k \in S}} a_S \prod_{\substack{i \in S \\ i \neq k}} x_i \right) \\ &= (-1)^{(x_k)} \sum_{\substack{(S, a_S) \in \text{Mon}(\mathcal{P}): \\ k \in S}} a_S \prod_{\substack{i \in S \\ i \neq k}} x_i \end{aligned}$$



## Bitflipmodifikation Funktionsauswertung

Funktionsauswertung für festes  $x \in M$  und  $k \in K$  also Summe über Elemente  $(S, a_S)$  aus  $\text{Mon}(\mathcal{P})$  mit  $k \in S$

$$(-1)^{(x_k)} \sum_{\substack{(S, a_S) \in \text{Mon}(\mathcal{P}): \\ k \in S}} a_S \prod_{\substack{i \in S \\ i \neq k}} x_i$$

Falls PBF mit n Variablen von Grad 2:  $|K| = |\text{Mon}(\mathcal{P}) : k \in S| = n$

Allgemein PBF mit Grad d:

$$|K| = |\text{Mon}(\mathcal{P}) : k \in S| = \sum_{i=0}^{d-1} \binom{n-1}{i}$$

Die meisten PBF Problemformulierungen verwenden Grad 2, sogenannte Quadratic unconstrained binary optimization QUBO.

Bei QUBO notwendige Monome zur Bitflippauswertung ist n, das heißt lineare Skalierung!

## PBF Problemformulierungen

Zahlenpartitionierung:

Teile Zahlen  $Z_1, \dots, Z_n$  in 2 Gruppen auf, sodass Differenz der Gruppen minimal ist.

Einfache Darstellung mit  $\{-1\}^n$ :

$$P(s) = \left( \sum_{i=1}^n Z_i s_i \right)^2$$

Partitionierung der Menge in negative und positiv addierte Zahlen. Quadratisierung  $\implies$  positiver Unterschied.

$$\min_{x \in \mathbb{F}_2^n} \left( \sum_{i=1}^n Z_i \frac{x_i + 1}{2} \right)^2$$

QUBO Formulierung mit voller Dichte  $\delta = 1$ .

## PBF Codierungen - Integer Linear Programm

ILP in Standardform:  $Ax = b, x \geq 0, \min c^T x, A \in \mathbb{R}^{m \times n}, b \in \mathbb{R}^m$

Als erstes Betrachtung Binary Linear Programm:  $x \in \mathbb{F}_2^n$

PBF-Modellierung als Penalty Terms:

$$P(x) = \alpha P_c(x) + \beta P_{A,b}(x)$$

Wobei

$$P_c(x) = \sum_{i=1}^n c_i x_i, \quad P_{A,b}(x) = \sum_{i=1}^m \left( \sum_{j=1}^n A_{i,j} x_j - b_i \right)^2$$

Lösung des (unbeschränkten) Hilfsproblems:

$$\min_{x \in M} c^T x + (Ax - b)^2$$

## PBF Codierungen - Integer Linear Programm

ILP in Standardform:  $Ax = b, g \geq x \geq 0, \min c^T x, A \in \mathbb{R}^{m \times n}, b \in \mathbb{R}^m$

$$P(x) = \alpha P_c(x) + \beta P_{A,b}(x)$$

mit

$$P_c(x) = \sum_{i=1}^n c_i x_i, \quad P_{A,b}(x) = \sum_{i=1}^m \left( \sum_{j=1}^n A_{i,j} x_j - b_i \right)^2$$

Skalierung der Parameter  $\alpha, \beta$ : Überlege kleinste/größte mögliche Werte für die Ausdrücke.

$C$  = Größter Wert von  $P_c(x)$

$D$  = Kleinster Wert von  $\beta P_{A,b}(x)$

Dann  $\implies C \cdot \alpha > D \cdot \beta$ .

Für ILP Substitution mit  $2^{t_{\max}} > g$

$$x_i = \sum_{t=1}^{t_{\max}} 2^t x_{i,t}.$$

QUBO Formulierung mit der Dichte  $\delta = 1$  abhängig von der Besetzung  $a_{i,j} \neq 0$  von  $A$ .

## PBF Codierungen - Max3SAT

Berühmtes Problem der Informatik:

Gegeben: Logischer Ausdruck  $B$  in  $x_1, \dots, x_n$  logischen Variablen verknüpft mit Operatoren  $\neg, \wedge, \vee$ .

Gesucht: Finde Belegung  $(x_1, \dots, x_n) \in \mathbb{F}_2^n$ , sodass Formel  $B$  erfüllt.

NP-vollständiges Problem!

SAT zu 3SAT Transformation: Jede Boolesche Formel  $B$  lässt sich ausdrücken durch  $z$  Klauseln  $C_t$  bestehend aus 3 negierten oder nicht negierten Variablen  $x_j, x_k, x_l$  verknüpft mit  $\vee$ .

$$B = C_1 \wedge \dots \wedge C_z \quad \text{mit} \quad C_t = \tilde{x}_i \vee \tilde{x}_j \vee \tilde{x}_k \quad \text{und} \quad \tilde{x}_i = \neg x_i \text{ oder } \tilde{x}_i = x_i$$

# PBF Codierungen - Max3SAT

Klauseln:

$$B = C_1 \wedge \dots \wedge C_z \quad \text{mit} \quad C_t = \tilde{x}_i \vee \tilde{x}_j \vee \tilde{x}_k \quad \text{und} \quad \tilde{x}_i = \neg x_i \quad \text{oder} \quad \tilde{x}_i = x_i$$

Codierung als PBF:

Repräsentiere boolesche Variablen durch  $x \in \mathbb{F}_2^n$ , konstruiere Polynome  $P$  mit

$$P(x_i, x_j, x_k) = 0 \iff C_t \text{ wahr}$$

$$P(x_i, x_j, x_k) = 1 \iff C_t \text{ falsch}$$

Das geht mit:

| Logischer Ausdruck                       | Einzige Falsche Belegung | Monome des PBF                                                    |
|------------------------------------------|--------------------------|-------------------------------------------------------------------|
| $(\neg x_i \vee \neg x_j \vee \neg x_k)$ | $(1,1,1)$                | $x_i x_j x_k$                                                     |
| $(x_i \vee \neg x_j \vee \neg x_k)$      | $(0,1,1)$                | $x_j x_k - x_i x_j x_k$                                           |
| $(x_i \vee x_j \vee \neg x_k)$           | $(0,0,1)$                | $x_k - x_i x_k - x_j x_k + x_i x_j x_k$                           |
| $(x_i \vee x_j \vee x_k)$                | $(0,0,0)$                | $1 - x_i - x_j - x_k + x_i x_k + x_i x_j + x_j x_k - x_i x_j x_k$ |

Ergibt PBF in  $n$  Variablen mit  $P(x) = 0 \iff B(x)$  wahr.

PBF hat Grad 3 und  $\delta(P)$  abhängig von der Zahl der Klauseln  $C_j$ .



# PBF Codierungen

Und viele mehr:

- ▶ 2-mean Clustering (Graphbisection)
- ▶ Join Ordering
- ▶ Portfolio Optimierung
- ▶ etc.

Google: Ising Form, PBF Form

Kontext Optimierung

Problemcodierung als Pseudo Boolsche Funktionen

Markow Ketten Perspektive

Implementation

# Stochastische Prozesse und Markowketten

Wie lassen sich die Algorithmen mathematisch beschreiben?

## Definition

Eine Familie von zufallsvariablen  $(X_t)_{t \in T}$  definiert auf einem Wahrscheinlichkeitstripel  $(M, \mathcal{A}, P_t)$  mit einer beliebigen Indexmenge heißt stochastischer Prozess. Die  $x \in M$  heißen Zustände der Markowkette.

Ist der Parameterraum  $T = \mathbb{N}$  und gilt die Markoweigenschaft

$$P_t(X_k = x_k | X_{k-1} = x_{k-1}, \dots, X_1 = x_1) = P_t(X_k = x_k | X_{k-1} = x_{k-1})$$

so heißt der stochastische Prozess Markowkette.

Ist diese Wahrscheinlichkeit  $P_t$  unabhängig von  $t$  so nennen wir die Markowkette homogen, anderenfalls inhomogen.

# Übergangswahrscheinlichkeiten SA

Wir betrachten SA für  $T$  konstant.

Algorithmus in Zustand  $x$ . Wahrscheinlichkeiten nur von aktuellem  $x$  abhängig.  
 Zweistufiges Zufallsexperiment: Wähle  $k \in K$  zufällig und dann Metropolis Kriterium

$$p_a(\Delta f(x)_{\theta_k}, T) = \min\{1, \exp(-\frac{\Delta f(x)_{\theta_k}}{T})\} = \exp(\frac{\{\max \Delta f(x)_{\theta_k}, 0\}}{T})$$

$$\Delta f(x)_{\theta_k}^+ := \max\{\Delta f(x)_{\theta_k}, 0\}$$

$\Rightarrow$  Übergangswahrscheinlichkeiten für nächsten Zustand  $y$ :

$$p_{x,y}^{SA} = \begin{cases} \frac{1}{|K|} \exp(-\Delta f(x)_{\theta_k}^+ / T) & , y \in N(x) \\ 1 - \sum_{i=1}^{|K|} \exp(-\Delta f(x)_{\theta_i}^+ / T) & , y = x \\ 0 & , y \notin N(x) \cup x \end{cases}$$

# Übergangswahrscheinlichkeiten DA

Ohne Escape Mechanismus. Keine Auswahl  $S$ ? erneute Schleife mit gleichem  $x$ .

Zweistufiges Zufallsexperiment: Auswahl einer Menge  $S$  mit  $k \in S$ . Danach gleichverteilte Auswahl aus der Menge  $S$ . Wahrscheinlichkeit dafür:

$$\frac{1}{|S|} \prod_{i \in S} \exp(-\Delta f(x)_{\theta_i}^+ / T) \prod_{i \in K/S} (1 - \exp(-\Delta f(x)_{\theta_i}^+ / T))$$

Summierte Wahrscheinlichkeiten für alle Möglichkeiten:

$$p_{x,y}^{DA} = \begin{cases} \sum_{\substack{S \subseteq K: \\ k \in S}} \frac{1}{|S|} \prod_{i \in S} \exp(-\Delta f(x)_{\theta_i}^+ / T) \prod_{i \in K/S} (1 - \exp(-\Delta f(x)_{\theta_i}^+ / T)) & , y = \theta_k(x) \in N(x) \\ \prod_{i=1}^{|K|} (1 - \exp(-\Delta f(x)_{\theta_i}^+ / T)) & , y = x \\ 0 & , y \notin N(x) \cup x \end{cases}$$

Andere Darstellung von  $p_{x,y}^{DA}$  mit  $y = \theta_k(x)$ :

$$\sum_{\substack{S \subseteq K: \\ k \in S}} \frac{1}{|S|} \prod_{i \in S} \exp(-\Delta f(x)_{\theta_i}^+ / T) \prod_{i \in K/S} (1 - \exp(-\Delta f(x)_{\theta_i}^+ / T))$$

Komplexe Null  $0 = \frac{1}{|K|} - \frac{1}{|K|}$  einführen, Summe aufteilen und  $\exp(-\Delta f(x)_{\theta_k}^+ / T)$  ausklammern.

$$\begin{aligned} & \sum_{\substack{S \subseteq K: \\ k \in S}} \left( \frac{1}{|S|} - \frac{1}{|K|} \right) \prod_{i \in S} \exp(-\Delta f(x)_{\theta_i}^+ / T) \prod_{i \in K/S} (1 - \exp(-\Delta f(x)_{\theta_i}^+ / T)) + \\ & \frac{1}{|K|} \exp(-\Delta f(x)_{\theta_k}^+ / T) \underbrace{\sum_{\substack{S \subseteq K: \\ k \notin S}} \prod_{i \in S} \exp(-\Delta f(x)_{\theta_i}^+ / T) \prod_{i \in (K/\{k\})/S} (1 - \exp(-\Delta f(x)_{\theta_i}^+ / T))}_{=1} \end{aligned}$$

Alle Zustände einer Multinomialverteilung:

$$\sim MULT(n-1, e^{(-\Delta f(x)_{\theta_1}^+ / T)}, \dots, e^{(-\Delta f(x)_{\theta_{k-1}}^+ / T)}, e^{(-\Delta f(x)_{\theta_{k+1}}^+ / T)}, \dots, e^{(-\Delta f(x)_{\theta_n}^+ / T)})$$

## Andere Darstellung der Übergangswahrscheinlichkeit

$$p_{x,y}^{DA} =$$

$$\underbrace{\frac{1}{|K|} \exp(-\Delta f(x)_{\theta_k}^+ / T)}_{\text{SA probability}} + \underbrace{\sum_{\substack{S \subseteq K: \\ k \in S}} \left( \frac{1}{|S|} - \frac{1}{|K|} \right) \prod_{i \in S} \exp(-\Delta f(x)_{\theta_i}^+ / T) \prod_{i \in K/S} (1 - \exp(-\Delta f(x)_{\theta_i}^+ / T))}_{\text{Additional DA probability}}$$

$$> p_{x,y}^{SA} = \frac{1}{|K|} \exp(-\Delta f(x)_{\theta_k}^+ / T)$$

Größere Übergangswahrscheinlichkeit als Simulated Annealing!

## Eigenschaften irreduzibel und aperiodisch

Notation:  $P_{x,y}^n :=$  Wahrscheinlichkeit in  $n$  Schritten von  $x$  zu  $y$  zu gelangen.

### Definition

Eine Markowkette ist irreduzibel, wenn für alle Zustände  $x, y$  und für mindestens ein  $n \in \mathbb{N}$  gilt:

$$P_{x,y}^n > 0$$

### Definition

Menge  $\{n \geq 1 : P_{i,i}^n > 0\}$  der Rückkehrmöglichkeiten für die Markowkette. Eine endliche Markowkette heißt aperiodisch, falls für alle Zustände  $i \in M$ :

$$\text{ggT}(\{n \geq 1 : P_{i,i}^n > 0\}) = 1$$

Erfüllt für beide Algorithmen, da:

- ▶ Irreduzibel: Erfüllt wegen Modifikationsabbildung, für zwei Elemente  $x, y$  existiert eine Sequenz  $k_1, \dots, k_n$  mit  $x = \theta_{k_1}(\dots \theta_{k_n}(y))$  in jeder Nachbarschaft existiert auch immer eine Wahrscheinlichkeit.
- ▶ Aperiodisch: Betrachte  $P_{i,i}^n$ , jeder Zustand kann in mindestens in 1 Schritt zum gleichen Zustand zurückkehren.  $\text{ggT}(\{n \geq 1 : P_{i,i}^n > 0\}) = 1$



# Stationäre Verteilungen von Markowketten

## Definition

Für eine Markowkette auf endlichem Zustandsraum, beschrieben durch eine einstufige Übergangsmatrix  $P_{x,y}$ , heißt der stochastische Vektor

$$\pi = (\pi_1, \pi_2, \dots) \quad \sum_{j \in M} \pi_j = 1$$

der folgende Gleichung erfüllt

$$\pi P = \pi$$

stationäre Verteilung der Markowkette.

## Satz (Quelle: Understanding Markowchains S. 170)

*Ist eine homogene Markowkette auf endlichem Zustandsraum irreduzibel und aperiodisch, so existiert eine eindeutig bestimmte stationäre Verteilung.*

*Außerdem konvergiert die Markowkette stochastisch gegen die stationäre Verteilung:*

$$\lim_{n \rightarrow \infty} P(X_n = i) = \pi_i$$

# Konvergenzuntersuchungen

Gesucht:

$$S^* = \{x \in M : f(x) = \min_{y \in M} f(y)\}$$

Satz (Simulated Annealing und Boltzmann Machines, Aarts und Vanderhoven, )

Die stationäre Verteilung der Markowkette definiert durch den SA Algorithmus ist die Boltzmannverteilung:

$$\pi_i = \lim_{n \rightarrow \infty} P(X_n = i) = \frac{1}{Z(T)} \exp(-f(i)/T) \quad \text{mit} \quad Z(T) = \sum_{i=1}^{|M|} \exp(-f(i)/T)$$

Folgerungen: Es gilt für die Markowkette des Simulated Annealing Algorithmus:

$$\lim_{n \rightarrow \infty} P(X_n = x) = \frac{1}{Z} \exp(-f(x)/T)$$

Mit Reduzierung von T werden die Ausdrücke  $\exp(-f(x)/T)$  größer je kleiner  $f(x)$  ist, also haben Minima größte Wahrscheinlichkeit! Also

$$\lim_{T \rightarrow 0} P(X_k \in S^*) \stackrel{?}{=} 1$$

Nicht im Allgemeinen. Konvergenz inhomogener Markowketten schwierig!

# Konvergenzbedingungen

Konvergenz stark abhängig von der Wahl des Cooling Schedules!

Lemma (aus Cooling Schedules for Optimal Annealing, Bruce Hajek (1988))

Für die inhomogenen Markowketten assoziiert mit dem SA und DA Algorithmus gilt

$$\lim_{T \rightarrow 0} \lim_{k \rightarrow \infty} P(X_k \in S^*) = 1$$

genau dann, wenn:

$$T_i = \frac{d}{\log(i)}$$

mit  $d$  der Tiefe des zweittiefsten lokalen Minimum.

Tiefe eines lokalen Minimum  $x$ :  $\partial f(x) - f(x)$

mit  $\partial f(x)$  Funktionswert ab dem lokales Minima wieder verlassen werden kann.

Bemerkungen:

Für Anwendungen schwierig, da Konvergenzaussage für unendlich viele Iterationen.

Außerdem muss man die Größe  $d$  kennen.



# Heuristiken

In der Anwendung sind die optimale Lösungen nicht immer notwendig.

**Heuristiken:** Verzicht auf Optimalität.

Generiere Lösungen über viele Monte Carlo Versuche, nimm davon die beste Lösung.

Kontext Optimierung

Problemcodierung als Pseudo Boolsche Funktionen

Markow Ketten Perspektive

Implementation

# Python Implementation

## Ziel:

Implementation der beiden Algorithmen Simulated Annealing und Digital Annealing in Python für beliebig codierte PBFs.

Relativer Vergleich der beiden Algorithmen.

## Python Implementation - Datentypen

PBF: endliche Menge von Paaren  $(S, a_S)$  von Monomen.  $\implies$  Speicherung als Dictionary

Dictionaries speichern Informationen in einer Menge von key und value Paaren. Der Aufruf

$$\text{value} = \text{dict}[\text{key}]$$

hat konstante Aufrufkomplexität  $\mathcal{O}(1)$  (Dictionaries sind Hash Maps!).

Speicherung eines PBFs durch Speicherung der Monome  $(S, a_S)$  als dictionary pbf:

- ▶ key: Tupel der Variablen des Monoms  $S \subseteq \{1, \dots, n\}$
- ▶ value: Koeffizient des Monoms  $a_S \in \mathbb{R}$

Speicherung der aktuellen Variablenbelegung  $x$  als dictionary varAssignment:

- ▶ key: Index der Variable  $i \in \{1, \dots, n\}$
- ▶ value: Wert der Variable  $x_i \in \mathbb{F}_2$



## Python Implementation - PBF Generierung

Beispiel an Zahlenpartitionierung:

$$\begin{aligned} \left(\sum_{i=1}^n Z_i s_i\right) &= \left(\sum_{i=1}^n Z_i (2x_i - 1)\right)^2 = \dots \\ &= 4 \sum_{i=1}^n \sum_{j=1}^n Z_i Z_j - 4 \sum_{i=1}^n \sum_{j=1}^n Z_i Z_j x_i + \sum_{i=1}^n \sum_{j=1}^n Z_i Z_j \end{aligned}$$

Monome hinzufügen.

## Algorithm Eval Pbf

```

1: function EVALPBF(pbf, varAssignment)
2:   Output = 0
3:   for mon in pbf do
4:     Impacts = True
5:     for k in mon do
6:       if varAssignment[k] = 0 then
7:         Impacts = False
8:         Break
9:       end if
10:    end for
11:    if Impacts = True then
12:      Output += pbf[mon]
13:    end if
14:  end for
15:  return Output
16: end function

```

Vorteil: Die Monomschleife wird bei der ersten 0 früher abgebrochen, ergibt Laufzeitgewinn. 

## Python Implementation - Einfache Teilfunktionen

- ▶ Modifiziere Variablenbelegung  $x$  mit  $k$ -Bitflip:

$$varAssignment = ModifyvarAssignment(varAssignment, k)$$

- ▶ Metropolis Kriterium
- ▶ Generierung von Zufallszahlen
- ▶ Generierung von Cooling Schedules
- ▶ Funktionsauswertung des Bitflips?

Mit Formel:

$$\Delta P(x)_{\beta_k} = (-1)^{(x_{k_1}+1)} \sum_{\substack{(S, a_S) \in \text{Mon}(\mathcal{P}): \\ k_1 \in S}} a_S \prod_{\substack{i \in S \\ i \neq k_1}} x_i$$

Erstelle dictionary pbfvardict:

- ▶ key: Index der Variable  $i \in (1, \dots, n)$
- ▶ value: Liste mit allen Monomen  $S$  mit  $i \in S$ .

Auswertung des  $k$ -Bitflips: Schleife über pbfvardict[k].

---

**Algorithm** Eval Delta Bitflip

```

1: function EVALDELTABITFLIP(pbf, varAssignment, pbfvardict, var)
2:   Output = 0
3:   for mon in pbfvardict[var] do
4:     Impacts = True
5:     for k in mon do
6:       if (varAssignment[k] = 0) and  $k \neq \text{var}$  then
7:         Impacts = False
8:         Break
9:       end if
10:    end for
11:    if Impacts = True then
12:      Output += pbf[mon]
13:    end if
14:  end for
15:  if varAssignment[var]=1 then
16:    Output = -Output
17:  end if
18:  return Output
19: end function

```

Vorteil: Skalierung mit  $\delta(PBF)$ !

## Python Implementation - Idee für Update deltaE vektor Methode

Problem: Immer noch sehr viel mehr extra Berechnungen für den Digital Annealing Algorithmus, denn nicht alle deltaE Anteile ändern sich nach einem Bitflip.

Wie verändert sich die schon berechneten  $\Delta P(x)_{\beta_k} \forall k \in |K|$ , wenn der Iterationsschritt Bitflip  $s$  durchgeführt wird.

$$\Delta P(x)_{\beta_{k_1}} = (-1)^{(x_k)} \sum_{\substack{(S, a_S) \in \text{Mon}(\mathcal{P}): \\ k \in S}} a_S \prod_{\substack{i \in S \\ i \neq k}} x_i$$

$$\Delta P(\beta_s(x))_{\beta_k} = ?$$

Position  $s$  wird mit  $-1$  multipliziert:

$$\Delta P(\beta_s(x))_{\beta_s} = -\Delta P(x)_{\beta_s}$$

Für restliche Änderungen betrachte Differenz der Ausdrücke!

Für  $k \neq s \in |K|$ :

$$\Delta P(\beta_s(x))_{\beta_k} - \Delta P(x)_{\beta_k} =$$

$$(-1)^{(x_k)} \sum_{\substack{(S, a_S) \in \text{Mon}(\mathcal{P}): \\ k \in S}} a_S \prod_{\substack{i \in S/\{s\} \\ i \neq k}} x_i \prod_{i \in S \cap \{s\}} (x_s \oplus 1) - (-1)^{(x_k)} \sum_{\substack{(S, a_S) \in \text{Mon}(\mathcal{P}): \\ k \in S}} a_S \prod_{\substack{i \in S \\ i \neq k}} x_i$$

Gleiche Monome kürzen sich weg:

$$(-1)^{(x_k)} \sum_{\substack{(S, a_S) \in \text{Mon}(\mathcal{P}): \\ k, s \in S}} a_S \prod_{\substack{i \in S/\{s\} \\ i \neq k}} x_i \prod_{i \in S \cap \{s\}} (x_s \oplus 1) - (-1)^{(x_k)} \sum_{\substack{(S, a_S) \in \text{Mon}(\mathcal{P}): \\ k, s \in S}} a_S \prod_{\substack{i \in S \\ i \neq k}} x_i =$$

$x_s$  ausklammern:

$$(x_s \oplus 1)((-1)^{(x_k)} \sum_{\substack{(S, a_S) \in \text{Mon}(\mathcal{P}): \\ k, s \in S}} a_S \prod_{\substack{i \in S/\{s\} \\ i \neq k, i \neq s}} x_i) - (x_s)((-1)^{(x_k)} \sum_{\substack{(S, a_S) \in \text{Mon}(\mathcal{P}): \\ k, s \in S}} a_S \prod_{\substack{i \in S/\{s\} \\ i \neq k, i \neq s}} x_i) =$$

$$(-1)^{(x_s)} (-1)^{(x_k)} \sum_{\substack{(S, a_S) \in \text{Mon}(\mathcal{P}): \\ k, s \in S}} a_S \prod_{\substack{i \in S \\ i \neq k, i \neq s}} x_i$$

Implementierung über Schleife über `pbfvardict[s]` und verteile obige Werte auf restliche  $k \in S$ .

---

**Algorithm** Update Delta E vector

---

```

1: function UPDATEDELTAVECTOR(deltaEvec, pbf, varAssignment, pbfvardict, var)
2:   deltaEvec[var]=-deltaEvec[var]
3:   for mon in pbfvardict[var] do
4:     for x in mon do
5:       if x  $\neq$  var then
6:         if varAssignment[var]=0 then
7:           varAssignemnt[var]=1
8:           E2 =EvalDeltaBitflip({mon: pbf[mon]},varAssignment,[mon], x)
9:           deltaEvec[x]+=E2
10:          varAssignemnt[var]=0
11:         else
12:           E2 =EvalDeltaBitflip({mon: pbf[mon]},varAssignment,[mon], x)
13:           deltaEvec[x]-=E2
14:         end if
15:       end if
16:     end for
17:   end for
18:   return deltaEvec
19: end function

```

---

Der Aufruf {mon: pbf[mon]} initialisiert ein einzelnes Monom dictionary.

---

**Algorithm** Efficient Digital Annealing algorithm with escape mechanism

---

```

1: function EFFICIENTDIGITALANNEALING(varAssignemnt,pbf,pbfvardict,T, $E_{inc}$ )
2:    $E = \text{Evalpbf}(pbf, \text{varAssignment})$ 
3:   for  $k = 1$  to  $|K|$  do
4:      $\text{deltaEvec}[k] = \text{EvalDeltaBitflip}(pbf, \text{varAssignment}, \text{pbfvardict}, k)$ 
5:   end for
6:   for  $i = 1$  to steps do
7:      $A = \text{leere Menge}$ 
8:      $T \leftarrow T_i$ 
9:     for  $k = 1$  to  $|K|$  do
10:       $p_i = \text{rand}(0, 1)$ 
11:      if  $p_i > \min(1, \exp((- \text{deltaEvec}[k] - E_{Off})/T))$  then
12:        Füge  $k$  der Menge  $A$  hinzu
13:      end if
14:    end for
15:    if  $A$  nicht leer then
16:       $E_{Off} = 0$ 
17:      Wähle zufälliges  $s$  aus  $A$ 
18:       $E += \text{deltaEvec}[s]$ 
19:       $\text{deltaEvec} = \text{UpdateDeltaEvector}(\text{deltaEvec}, pbf, \text{varAssignment}, \text{pbfvardict}, s)$ 
20:       $\text{varAssignement} = \text{ModifyvarAssignment}(\text{varAssignement}, s)$ 
21:    else
22:       $E_{Off} = E_{Off} + E_{inc}$ 
23:    end if
24:  end for
25:  return  $x, f(x)$ 
26: end function

```

---

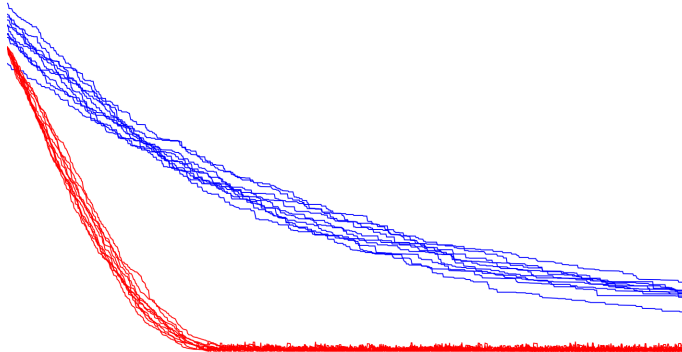




## Python Implementation - Tests

Zuerst zufällig generiertes QUBO PBF.

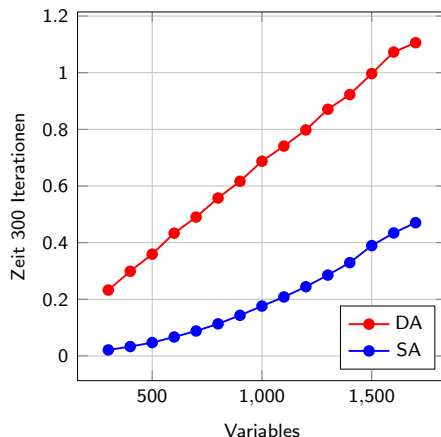
Vergleich der Trajektorien: DA rot, SA blau.



SA hat die langsam werdende Konvergenzgeschwindigkeit. Schnellere Konvergenzgeschwindigkeit bei DA!

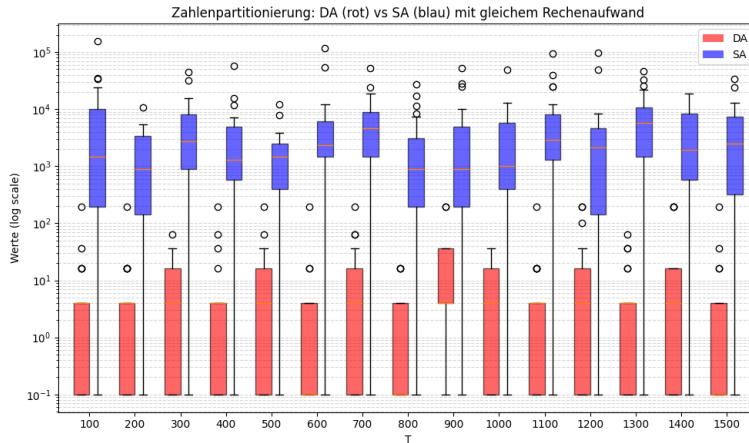
Approximation der nötigen Rechenzeit pro Iteration abhängig von der Größe der Variablen. Jeweils gemittelt über 25 Monte Carlo Trials und 10 Probleminstanzen.

| n    | Monome  | Zeit $f(x)$ | Zeit $\Delta f(x)_\theta$ |
|------|---------|-------------|---------------------------|
| 300  | 45150   | 0.004517    | 0.003631                  |
| 400  | 80200   | 0.008405    | 0.008663                  |
| 500  | 125250  | 0.012771    | 0.010039                  |
| 600  | 180300  | 0.020381    | 0.015336                  |
| 700  | 245350  | 0.027177    | 0.021451                  |
| 800  | 320400  | 0.038791    | 0.028558                  |
| 900  | 405450  | 0.048784    | 0.036946                  |
| 1000 | 500500  | 0.062148    | 0.045597                  |
| 1100 | 605550  | 0.076533    | 0.057710                  |
| 1200 | 720600  | 0.091012    | 0.068728                  |
| 1300 | 845650  | 0.110055    | 0.078901                  |
| 1400 | 980700  | 0.131725    | 0.093468                  |
| 1500 | 1125750 | 0.157204    | 0.108929                  |
| 1600 | 1280800 | 0.192059    | 0.128441                  |
| 1700 | 1445850 | 0.206326    | 0.138029                  |



## Python Implementation - Tests

Zahlenpartitionierung mit 1500 Zahlen und mit Rechenzeit SA = Rechenzeit DA über  
Iterationen SA \* Faktor = Iterationen DA



## Quintessenz

- ▶ Beide Heuristiken sehr nützlich, da für viele Probleme anwendbar (universell).
- ▶ Mit einer effizienten Implementierung ist DA besser, das ist aber auch Problemabhängig.
- ▶ Weitere Untersuchungen zur Konvergenz notwendig!

# Danke für die Aufmerksamkeit!

# Literaturverzeichnis

Siehe pdf.